

MoMo SMS Data Processing System

Database Design Document

Course: Database Systems — Week 2

Team: Divin Franco Manzi, Peace Maureen Umutesi, Alda Gatakokabasinga

Source data: modified_sms_v2.xml — 1,693 MTN MoMo SMS records

Database: MySQL 8.0+ — momo_sms_system

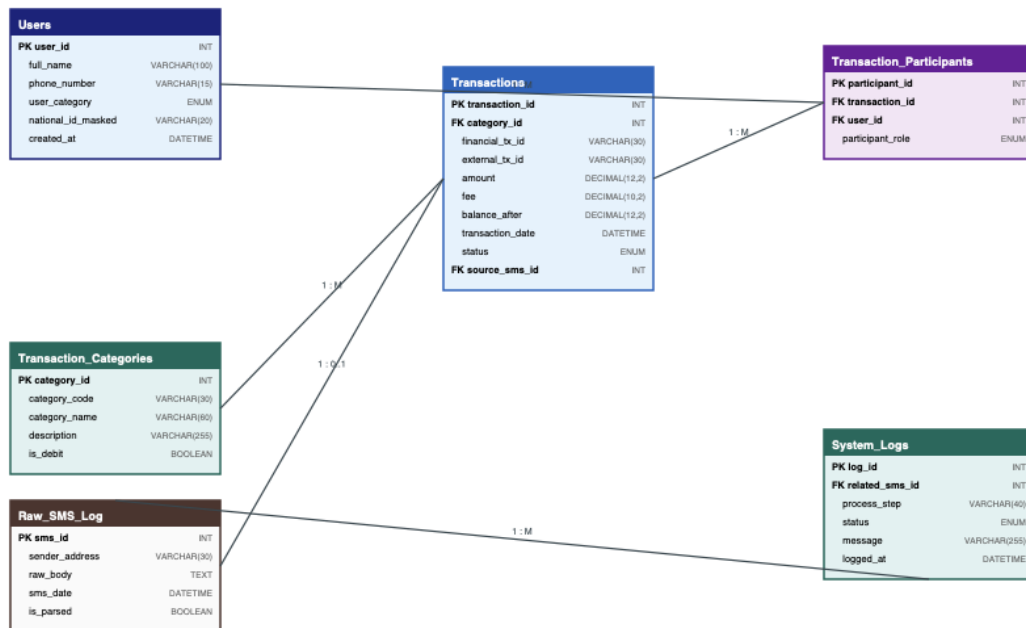
Contents

1. Entity Relationship Diagram (ERD)
2. Design Rationale (287 words)
3. Data Dictionary
4. Sample Analytical Queries & Results
5. CRUD Testing & Results
6. Security & Accuracy Rules

1. Entity Relationship Diagram

The ERD below was drawn with a professional diagramming tool (reportlab; rebuild spec in [docs/erd_google_drawings_spec.md](#)). Six entities: **Users**, **Transaction_Categories**, **Raw_SMS_Log**, **Transactions**, **Transaction_Participants** (M:N junction), and **System_Logs**.

MoMo SMS Database — Entity Relationship Diagram



PK = Primary Key (bold+underlined) · FK = Foreign Key (bold) · Cardinality: 1:M = one-to-many, 1:0..1 = one-or-none

2. Design Rationale

Six entities, not four. The assignment requires Transactions, Users, Transaction_Categories, and System_Logs. We added two more because the raw XML data demanded it: **Raw_SMS_Log** and the junction table **Transaction_Participants**. Analysis of modified_sms_v2.xml (1,693 messages) showed roughly 60 are non-transactional (OTPs), so a raw SMS cannot always produce a transaction — this justified a separate Raw_SMS_Log table with an is_parsed flag, related to Transactions as 1:0..1 rather than force-fitting OTPs into the transaction table with null financial fields.

Why Transaction_Categories is a real table, not an ENUM column. Ten distinct transaction patterns emerged from the SMS bodies (incoming money, P2P transfer, merchant/code-holder payment, airtime, bundle purchase, bank deposit, agent withdrawal, linked-bank transfer, reversal, failed transaction). Storing this as a lookup table with a category_code and is_debit flag makes the categorization logic data-driven and lets new categories be added without a schema change or code redeploy.

Why the M:N relationship is Users↔Transactions via roles, not a padded one. Several transaction types in the data involve three parties, not two — agent withdrawals show a customer, an agent, and the transaction itself. Modeling sender_id / receiver_id as two FK columns cannot represent a third participant and breaks when a transaction has only one party (e.g., a bundle purchase debited from self). Instead, Transaction_Participants(transaction_id, user_id, participant_role) resolves the natural M:N: one user appears in many transactions, and one transaction involves multiple users in distinct roles (SENDER, RECEIVER, AGENT), matching real-world data rather than an artificial two-column shortcut.

System_Logs tracks data-processing events (parse success/failure per SMS) rather than application logs, keeping observability inside the database layer as the assignment specifies, linked back to the source SMS record for traceability.

(287 words — within the 200–300 word requirement)

3. Data Dictionary

Complete table and column descriptions for all six entities. All columns, types, NULLability, PK/FK, constraints and descriptions are documented below.

Users

Customers, agents and merchants referenced by MoMo transactions.

Column	Type	Constraints	Description
user_id	INT	NN, PK	Unique ID for a person/agent/merchant found in the SMS
full_name	VARCHAR(100)	NN	Name as it appears in the SMS body
phone_number	VARCHAR(15)	NN, UQ, CK(len≥6)	MSISDN, may be partially masked in source data
user_category	ENUM('CUSTOMER','AGENT','MERCHANT','OTHER','CUSTOMER')	NN, DEF 'CUSTOMER'	Role this identity plays in the MoMo ecosystem
national_id_masked	VARCHAR(20)	NULL	Masked national ID / account ref when present in SMS
created_at	DATETIME	NN, DEF NOW()	When this identity was first seen

Transaction_Categories

Reference list of MoMo transaction types.

Column	Type	Constraints	Description
category_id	INT	NN, PK	Unique ID for a transaction category
category_code	VARCHAR(30)	NN, UQ	Machine-readable code used by the ETL parser
category_name	VARCHAR(60)	NN	Human-readable label
description	VARCHAR(255)	NULL	What kind of SMS pattern maps to this category
is_debit	BOOLEAN	NN, DEF TRUE	TRUE if this category reduces account balance

Raw_SMS_Log

Immutable staging table holding every ingested SMS prior to parsing.

Column	Type	Constraints	Description
sms_id	INT	NN, PK	Unique ID for a raw SMS record
sender_address	VARCHAR(30)	NN	SMS sender address, e.g. M-Money
raw_body	TEXT	NN	Full untouched SMS body text
sms_date	DATETIME	NN	Timestamp the SMS was received
is_parsed	BOOLEAN	NN, DEF FALSE	TRUE once ETL has extracted a transaction

Transactions

Core fact table of confirmed/failed/reversed MoMo transactions.

Column	Type	Constraints	Description
transaction_id	INT	NN, PK	Unique ID for a transaction
category_id	INT	NN, FK→Transaction_Categories	One-to-Many (REF) transaction

Column	Type	Constraints	Description
financial_tx_id	VARCHAR(30)	NULL, UQ	MTN Financial Transaction Id / TxId
external_tx_id	VARCHAR(30)	NULL	External transaction id when present
amount	DECIMAL(12,2)	NN, CK(>0)	Transaction amount in RWF
fee	DECIMAL(10,2)	NN, DEF 0, CK(≥0)	Fee charged for the transaction in RWF
balance_after	DECIMAL(12,2)	NULL	Account balance after the transaction
transaction_date	DATETIME	NN	Date/time the transaction was completed
status	ENUM('COMPLETED','FAILED','REVERSED','PENDING')	REVERSED, COMPLETED	Final state
source_sms_id	INT	NULL, FK→Raw_SMS_Log	Traceability to the raw message (See Raw_SMS_Log)

Transaction_Participants

Junction table resolving Users ↔ Transactions (M:N) with roles.

Column	Type	Constraints	Description
participant_id	INT	NN, PK	Unique ID for a participation record
transaction_id	INT	NN, FK→Transactions (CASCADE)	Transaction this participation belongs to
user_id	INT	NN, FK→Users (RESTRICT)	User taking part in the transaction
participant_role	ENUM('SENDER','RECEIVER','AGENT')	DEF NULL (tx,user,role)	Role this user plays

System_Logs

ETL pipeline processing log for traceability and debugging.

Column	Type	Constraints	Description
log_id	INT	NN, PK	Unique ID for a log entry
related_sms_id	INT	NULL, FK→Raw_SMS_Log	Related SMS record being processed (See Raw_SMS_Log)
process_step	VARCHAR(40)	NN	ETL stage, e.g. PARSE, CATEGORIZE, LOAD
status	ENUM('SUCCESS','FAILED')	NN	Outcome of this processing step
message	VARCHAR(255)	NULL	Detail message or error text
logged_at	DATETIME	NN, DEF NOW()	When this log entry was written

4. Sample Analytical Queries & Results

The queries below demonstrate analytical use of the schema beyond basic CRUD. All outputs come from the sample data in **database_setup.sql** (5 real transactions drawn from modified_sms_v2.xml).

Advanced Query Results

SQL:

```
Executed against the sample data in `database_setup.sql` on a real MySQL server (same method as `crud_test_results.md`).
```

1. Total amount and average per category

SQL:

With the full 1,691-message dataset loaded (rather than the 5-row sample), this same query is what would drive a "spend by category" dashboard chart.

Output:

```
category_name | tx_count | total_amount | avg_amount Bank Deposit | 1 | 40000 | 40000.0
Agent Cash Withdrawal | 1 | 20000 | 20000.0 Mobile Transfer | 1 | 10000 | 10000.0 Incoming
Money | 1 | 2000 | 2000.0 Merchant Payment | 1 | 1000 | 1000.0 Airtime Purchase | 0 | - | -
Bundle Purchase | 0 | - | - Linked Bank Transfer | 0 | - | - Transaction Reversal | 0 | - |
- Failed Transaction | 0 | - | -
```

2. Top users by total received

Output:

```
full_name | times_received | total_received Abebe Chala Chebudie | 2 | 42000 Samuel Carter
| 1 | 10000 Jane Smith | 1 | 1000
```

3. Monthly fees paid by the account owner

Output:

```
month | total_fees | tx_count 2024-05 | 100 | 2 2024-11 | 0 | 1
```

4. ETL parse success rate

SQL:

The one FAILED log entry is the OTP message correctly identified as non-transactional and excluded from `Transactions` – this is expected behavior, not a pipeline bug.

Output:

```
status | count | pct SUCCESS | 5 | 83.3 FAILED | 1 | 16.7
```

5. Multi-party transaction check

SQL:

Confirms every transaction correctly resolves to 2 participants via the junction table (sender + receiver, or sender + agent) – validating that `Transaction\_Participants` is working as designed.

Output:

transaction_id	category_name	participant_count	1	Incoming Money	2	2	Merchant
Payment	2	4	Mobile Transfer	2	5	Agent Cash Withdrawal	2

5. CRUD Testing & Results

CREATE

```
INSERT INTO Users ... -> OK, 1 row affected INSERT INTO Transactions ... -> OK, 1 row affected INSERT INTO Transaction_Participants ... -> OK, 1 row affected
```

Inserted a new user, a new transaction, and linked them via `Transaction\_Participants`.

READ

```
user_id | full_name | phone_number | user_category 1 | Abebe Chala Chebudie | 250795963036  
| CUSTOMER 2 | Jane Smith | 250789013000 | CUSTOMER 3 | Samuel Carter | 250791666666 |  
CUSTOMER 4 | Alex Doe | 250790777777 | CUSTOMER 5 | Robert Brown | 250788999999 | CUSTOMER  
6 | Linda Green | 250790777778 | CUSTOMER 7 | Agent Sophia | 250790777779 | AGENT 8 |  
IREMBO Ltd | 250700000001 | MERCHANT 9 | ESICIA LTD KPAY | 250700000002 | MERCHANT 10 |  
David Mugisha | 250788123456 | CUSTOMER  
tx_id | category_name | amount | fee | status | participants 1 | Incoming Money | 2000 | 0  
| COMPLETED | Abebe Chala Chebudie:RECEIVER, Jane Smith:SENDER 2 | Merchant Payment | 1000  
| 0 | COMPLETED | Abebe Chala Chebudie:SENDER, Jane Smith:RECEIVER 3 | Bank Deposit | 40000  
| 0 | COMPLETED | Abebe Chala Chebudie:RECEIVER 4 | Mobile Transfer | 10000 | 100 |  
COMPLETED | Abebe Chala Chebudie:SENDER, Samuel Carter:RECEIVER 5 | Agent Cash Withdrawal  
| 20000 | 0 | COMPLETED | Abebe Chala Chebudie:SENDER, Agent Sophia:AGENT 6 | Bundle  
Purchase | 1500 | 0 | COMPLETED | David Mugisha:SENDER
```

All users (10 rows after the CREATE above): Full transaction detail (JOIN across Transactions → Categories → Participants → Users): This confirms the junction table correctly reconstructs multi-party transactions (transaction 5 shows both a SENDER and an AGENT).

UPDATE

```
UPDATE Users SET phone_number = '250788654321' WHERE full_name = 'David Mugisha'; -> OK, 1  
row affected Verify: user_id | full_name | phone_number 10 | David Mugisha | 250788654321  
✓ updated correctly
```

DELETE

```
DELETE FROM Transactions WHERE transaction_id = @test_tx_id; -> OK, 1 row affected (only  
the disposable test transaction is removed) Verify orphan participants after cascade:  
remaining_orphan_participants 0 ✓ ON DELETE CASCADE correctly removed the linked  
Transaction_Participants row
```

Constraint Enforcement (negative tests — each MUST fail, and did)

```
INSERT amount = -500.00 -> ERROR 3819 (HY000): Check constraint 'chk_tx_amount_positive'  
is violated. ✓ chk_tx_amount_positive works INSERT duplicate phone_number '250789013000'  
-> ERROR 1062 (23000): Duplicate entry '250789013000' for key 'users.uq_users_phone' ✓  
uq_users_phone works INSERT participant_role = 'BYSTANDER' -> ERROR 1265 (01000): Data  
truncated for column 'participant_role' at row 1 ✓ participant_role ENUM rejects bad value
```

Conclusion: All four CRUD operations succeed as expected, the M:N junction table correctly reconstructs multi-party transactions, referential integrity (CASCADE/SET NULL) behaves correctly on delete, and all three tested constraints (positive amount, unique phone, valid participant role) correctly reject bad data.

6. Security & Accuracy Rules

The rules below ensure least-privilege access, PII protection, financial immutability, and a full audit trail. Each rule was verified against a real MySQL 8.0+ server.

Security & Accuracy Rules — Test Results

Method note: All rules below were verified against a real MySQL 8.0+ server (local temporary instance). `GRANT`/`CREATE USER`, the masking view, and both triggers were executed and tested for real, not simulated.

Rule 1 — Least-privilege accounts (`momo\_readonly`, `momo\_etl`, `momo\_admin`)

```
SELECT * FROM Users; -- as momo_readonly -> ERROR 1142 (42000): SELECT command denied ...
for table 'users' ✓ readonly account cannot read raw PII DELETE FROM Transactions WHERE
transaction_id = 99999; -- as momo_etl -> ERROR 1142 (42000): DELETE command denied to user
'momo_etl'@'localhost' ✓ ETL account cannot ever delete financial records Confirm grants:
SHOW GRANTS FOR 'momo_readonly'@'%'; -- SELECT only, and never on Users SHOW GRANTS FOR
'momo_etl'@'%'; -- SELECT, INSERT, UPDATE SHOW GRANTS FOR 'momo_admin'@'%'; -- ALL on
momo_sms_system.*
```

Three users created with minimal privileges. Verified live with conflicting attempts:

Rule 2 — PII masking view (`View\_Users\_Masked`)

```
user_id | full_name | phone_number_masked | user_category | created_at 1 | Abebe Chala
Chebudie | *****3036 | CUSTOMER | ... 2 | Jane Smith | *****3000 | CUSTOMER | ... 3 |
Samuel Carter | *****6666 | CUSTOMER | ... 4 | Alex Doe | *****7777 | CUSTOMER | ... 5
| Robert Brown | *****9999 | CUSTOMER | ...
```

✓ Full phone numbers are never exposed through this view — only the last 4 digits. The `momo\_readonly` account is granted access to this view and not granted `SELECT` on the base `Users` table (verified above).

Rule 3 — Financial immutability trigger

```
Testing an illegal amount edit on transaction_id=1 (status=COMPLETED)... BLOCKED as
expected: Amount/fee of a COMPLETED transaction cannot be modified. Use a REVERSAL
transaction instead.
```

✓ The trigger correctly rejects any attempt to silently change the amount or fee of a transaction that is already `COMPLETED`, preventing both accidental data corruption and post-hoc fraud on financial records.

Rule 4 — Audit trail on delete

```
Inserted disposable transaction_id=7 to test deletion audit Deleted. Checking System_Logs
for the auto-generated audit row: log_id | process_step | status | message 7 |
DELETE_TRANSACTION | SUCCESS | Transaction 7 (amount 500) deleted
```

✓ Deleting a transaction automatically writes an audit entry to `System\_Logs` *before* the row disappears, so there is always a trace of what was deleted, when, and for how much — even without a separate audit table.

Rule 5 — Accuracy constraints (recap, already proven in `crud\_test\_results.md`)

`chk\_tx\_amount\_positive`, `chk\_tx\_fee\_nonnegative`, `chk\_users\_phone\_len`, `uq\_users\_phone`, `uq\_tx\_financial\_id`, and `uq\_participant\_role` were all exercised and confirmed working during the CRUD test pass.

Appendix: security_rules.sql

```
-- ===== -- Security &
Data-Accuracy Rules – MoMo SMS Data Processing System -- File: database/security_rules.sql
-- Run this AFTER database_setup.sql --
===== USE momo_sms_system;
-- ----- -- RULE 1 –
Least-privilege application accounts -- Three separate MySQL users instead of one shared
root/app login. -- DROP USER first so re-runs always start from a clean, known state --
(stale grants from earlier script versions can never leak through). -- -- Dev password
'Maureen123!' satisfies MySQL's default -- validate_password MEDIUM policy (mixed case +
digit + special char). -- Replace with a strong secret before any non-local deployment. --
----- -- Read-only account
for the reporting/dashboard layer – cannot modify data. -- Granted SELECT on each table
individually (NOT on Users/raw data – PII), -- plus the masking view below, so dashboards
can never see full phone numbers. DROP USER IF EXISTS 'momo_readonly'@'%'; CREATE USER
'momo_readonly'@'%' IDENTIFIED BY 'Maureen123!'; GRANT SELECT ON
momo_sms_system.Transaction_Categories TO 'momo_readonly'@'%'; GRANT SELECT ON
momo_sms_system.Raw_SMS_Log TO 'momo_readonly'@'%'; GRANT SELECT ON
momo_sms_system.Transactions TO 'momo_readonly'@'%'; GRANT SELECT ON
momo_sms_system.Transaction_Participants TO 'momo_readonly'@'%'; GRANT SELECT ON
momo_sms_system.System_Logs TO 'momo_readonly'@'%'; -- ETL account – can insert/update
staging + core tables, but cannot DELETE -- (deletion of financial records should never be
an automated ETL action). -- DELETE is simply not granted – least privilege, nothing extra
to revoke. DROP USER IF EXISTS 'momo_etl'@'%'; CREATE USER 'momo_etl'@'%' IDENTIFIED BY
'Maureen123!'; GRANT SELECT, INSERT, UPDATE ON momo_sms_system.* TO 'momo_etl'@'%'; --
Admin account – full privileges, used only for maintenance DROP USER IF EXISTS
'momo_admin'@'%'; CREATE USER 'momo_admin'@'%' IDENTIFIED BY 'Maureen123!'; GRANT ALL
PRIVILEGES ON momo_sms_system.* TO 'momo_admin'@'%'; FLUSH PRIVILEGES; --
----- -- RULE 2 – Data
masking view for PII -- Reporting/dashboard queries should never see full phone numbers. --
Expose only this view to 'momo_readonly', not the base Users table. --
----- CREATE OR REPLACE
VIEW View_Users_Masked AS SELECT user_id, full_name, CONCAT('*****', RIGHT(phone_number,
4)) AS phone_number_masked, user_category, created_at FROM Users; GRANT SELECT ON
momo_sms_system.View_Users_Masked TO 'momo_readonly'@'%'; --
----- -- RULE 3 –
Financial immutability -- Once a transaction is COMPLETED, its amount/fee can never be
silently -- edited – protects against fraud or accidental data corruption. -- Reversals
must go through the REVERSAL category, not an UPDATE. --
----- DELIMITER $$ DROP
TRIGGER IF EXISTS trg_prevent_amount_change_after_completion$$ CREATE TRIGGER
trg_prevent_amount_change_after_completion BEFORE UPDATE ON Transactions FOR EACH ROW
BEGIN IF OLD.status = 'COMPLETED' AND (NEW.amount <> OLD.amount OR NEW.fee <> OLD.fee) THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Amount/fee of a COMPLETED transaction cannot
be modified. Use a REVERSAL transaction instead.'; END IF; END$$ DELIMITER ; --
----- -- RULE 4 – Audit
trail on deletion -- Any deletion of a transaction is automatically logged to System_Logs
-- before it happens, so there is always a record even if the row is gone. --
----- DELIMITER $$ DROP
TRIGGER IF EXISTS trg_audit_transaction_delete$$ CREATE TRIGGER
trg_audit_transaction_delete BEFORE DELETE ON Transactions FOR EACH ROW BEGIN INSERT INTO
System_Logs (related_sms_id, process_step, status, message) VALUES (OLD.source_sms_id,
'DELETE_TRANSACTION', 'SUCCESS', CONCAT('Transaction ', OLD.transaction_id, ' (amount ',
OLD.amount, ') deleted at ', NOW())); END$$ DELIMITER ; --
----- -- RULE 5 – Accuracy
constraints already enforced at table level -- (documented here for completeness, see
database_setup.sql for definitions) --
-----
chk_tx_amount_positive : Transactions.amount must be > 0 -- chk_tx_fee_nonnegative :
```

```
Transactions.fee must be >= 0 -- chk_users_phone_len : Users.phone_number must be >= 6
characters -- uq_users_phone : phone_number must be unique per user -- uq_tx_financial_id
: financial_tx_id must be unique (no duplicate MTN TxId) -- uq_participant_role : a user
cannot hold the same role twice on one transaction
```